

Manual de JUnit 6 (Jupiter)

Table of Contents

1. ¿Qué es JUnit?
 - 1.1. Dependencias Maven
2. Anatomía de un test
3. Aserciones
 - 3.1. Comprobar varias cosas a la vez: `assertAll`
4. Ciclo de vida de un test
 - 4.1. Una instancia por test (por defecto)
 - 4.2. Métodos de ciclo de vida
5. Probar que algo lanza una excepción
6. Comprobar que algo no tarda demasiado
 - 6.1. La alternativa declarativa: `@Timeout`
7. Tests parametrizados
 - 7.1. `@ValueSource` : una lista de literales
 - 7.2. `@MethodSource` : un método que genera los datos
 - 7.3. Otras fuentes disponibles
8. Desactivar un test: `@Disabled`
9. Precondiciones (`Assumptions`)
10. Ejecución en paralelo
11. Extensiones (`@ExtendWith`)
12. Buenas prácticas (y errores comunes)
13. Referencia rápida

Manual de referencia rápida sobre JUnit 6 (la API Jupiter). No documenta un ejercicio concreto: recoge, organizados por concepto, los mecanismos de JUnit más usados en el día a día, con ejemplos de código propios.

1. ¿Qué es JUnit?

JUnit es el framework de referencia para escribir y ejecutar **pruebas unitarias** en Java. En lugar de comprobar el comportamiento de una clase a mano (con un `main()` y un `if / else` que imprime `OK / ERROR`), JUnit aporta:

- Un conjunto de anotaciones (`@Test` , `@BeforeEach` ...) para declarar qué es un test y cuándo debe ejecutarse cada pieza de código alrededor de él.
- Una librería de aserciones (`assertEquals` , `assertThrows` ...) que compara el resultado real con el esperado y hace fallar el test — con un mensaje claro — cuando no coinciden.
- Un motor que descubre los tests automáticamente (sin que nadie tenga que llamarlos uno a uno), los ejecuta y genera un informe.

- Integración con Maven/Gradle y con los IDEs, de forma que "ejecutar los tests" es una acción de un solo clic o un único comando (`mvn test`).

JUnit 5 (y su continuación, JUnit 6) se organiza en tres subproyectos:

Módulo	Para qué sirve
JUnit Platform	La base: define cómo se descubren y lanzan los tests, y permite que herramientas de terceros (Maven, Gradle, IDEs) hablen un mismo lenguaje independientemente del framework de test que haya detrás.
JUnit Jupiter	El framework de test "moderno": las anotaciones y aserciones que se usan en el día a día (<code>@Test</code> , <code>@BeforeEach</code> , <code>Assertions</code> ...). Es el que se documenta en este manual.
JUnit Vintage	Un motor de compatibilidad para poder seguir ejecutando tests antiguos escritos con JUnit 3/4 sobre la Platform moderna.

1.1. Dependencias Maven

Para escribir y ejecutar tests con Jupiter hacen falta, como mínimo, dos artefactos con `scope=test` :

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>6.0.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>6.0.3</version>
  <scope>test</scope>
</dependency>
```

XML

`junit-jupiter-api` trae las anotaciones y las aserciones (lo que se usa al **escribir** el test); `junit-jupiter-engine` es el motor que Maven/el IDE usan en tiempo de ejecución para **descubrir y correr** esos tests — sin él, el código compila pero no se ejecuta nada. Si además se van a usar tests parametrizados hace falta un tercer artefacto:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-params</artifactId>
  <version>6.0.3</version>
  <scope>test</scope>
</dependency>
```

XML

Con Maven, si los tests no siguen el convenio de nombres que espera **Surefire** por defecto (`/Test*.java` , `/*Test.java` , `/*Tests.java` , `/*TestCase.java`), `mvn test` no los ejecutará aunque el IDE sí los detecte. Se puede ampliar la búsqueda añadiendo un `<include>` en `maven-surefire-plugin`:



```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <includes>
      <include>**/ejemplos/**</include>
    </includes>
  </configuration>
</plugin>
```

XML

2. Anatomía de un test

```
public class CalculadoraTest {

    @Test
    @DisplayName("La suma de dos números debe ser correcta")
    void laSumaEsCorrecta() {
        System.out.println("Ejecutando el test");
    }

}
```

JAVA

- La clase de test necesita **constructor por defecto** (sin argumentos): JUnit crea la instancia por sí solo, no hay forma de pasarle parámetros al construirla.
- Un método se convierte en test anotándolo con `@Test` . Tiene que ser `void` , sin parámetros (salvo que se use inyección de parámetros, como en los tests parametrizados) y no necesita ser `public` a partir de JUnit 5 (aunque `public` sigue siendo lo más habitual).
- `@DisplayName` es opcional: da un nombre legible al test en los informes, útil cuando el nombre del método no es lo bastante descriptivo por sí mismo.
- Por defecto, JUnit ejecuta los métodos `@Test` de una clase en un orden **determinista pero no declarado** (no es, en general, el orden en que están escritos en el fichero). Si el orden importa para algo, es una señal de que los tests están acoplados entre sí — hay que evitarlo (ver Ciclo de vida de un test) o, si de verdad hace falta forzar un orden, usar `@TestMethodOrder` .

3. Aserciones

Las aserciones (clase `org.junit.jupiter.api.Assertions`) comparan lo que ha pasado con lo que se esperaba. Si no coincide, lanzan una excepción (`AssertionFailedError`) que JUnit captura y convierte en un test en rojo.

Aserto	Para qué
<code>assertEquals(esperado, real)</code>	Compara con <code>equals()</code> . El más usado con diferencia.

Aserto	Para qué
<code>assertNotEquals(a, b)</code>	Lo contrario. Admite un mensaje de error opcional como último parámetro.
<code>assertTrue(expr)</code> / <code>assertFalse(expr)</code>	Comprueba una expresión booleana cualquiera — sirve para cualquier condición que no encaje en otro aserto más específico.
<code>assertNull(ref)</code> / <code>assertNotNull(ref)</code>	Comprueba si una referencia es null.
<code>assertSame(a, b)</code> / <code>assertNotSame(a, b)</code>	Compara con <code>==</code> (identidad de objeto), no con <code>equals()</code> .
<code>assertArrayEquals(a, b)</code>	Compara dos arrays elemento a elemento.
<code>assertIterableEquals(a, b)</code>	Igual que el anterior, pero para cualquier <code>Iterable</code> (listas, sets...).
<code>assertThrows(TipoExcepcion.class, ejecutable)</code>	Comprueba que el código lanza la excepción indicada. Ver Probar que algo lanza una excepción.
<code>assertTimeoutPreemptively(duracion, ejecutable)</code>	Comprueba que el código no tarda más de lo indicado. Ver Comprobar que algo no tarda demasiado.
<code>Assertions.fail(mensaje)</code>	Hace fallar el test explícitamente. Útil cuando la condición de fallo es difícil de expresar como una expresión booleana, o para marcar temporalmente un test como pendiente.
<code>Assertions.assertAll(ejecutable...)</code>	Agrupar varios asertos independientes. Ver más abajo.



Cuidado con `assertSame` / `==` en tipos envoltorio (`Integer`, `Long`...): Java cachea las instancias de `Integer` entre -128 y 127 (*Integer cache*), así que `Integer.valueOf(127) == Integer.valueOf(127)` es `true`, pero `Integer.valueOf(128) == Integer.valueOf(128)` ya no lo es. Para comparar **valores** (que es casi siempre lo que se quiere) hay que usar `equals()` / `assertEquals`, nunca `assertSame` / `==`, con envoltorios.



Comparar `double` / `float` con `==` (o `assertTrue(a == b)`) es frágil: los errores de redondeo de coma flotante hacen que dos cálculos "matemáticamente iguales" puedan no serlo a nivel de bits. `assertEquals(esperado, real)` con `double` es la forma segura, y si hace falta admitir un margen de error existe una variante de `assertEquals` con un parámetro `delta`.

3.1. Comprobar varias cosas a la vez: `assertAll`

Con asertos sueltos, en cuanto uno falla se lanza la excepción correspondiente y el resto del método no se llega a ejecutar — si hay tres asertos y falla el primero, nunca se sabrá si los otros dos también habrían fallado:

```
assertNotNull(resultado);
assertTrue(resultado > 10);    // si esto falla, la siguiente línea no se ejecuta
assertTrue(resultado < 100);
```

`Assertions.assertAll(...)` ejecuta todas las comprobaciones aunque alguna falle, y el informe final lista **todas** las que no se cumplieron:

```
Assertions.assertAll(  
    () -> assertNotNull(resultado, "El resultado no puede ser nulo"),  
    () -> assertTrue(resultado > 10, "Debería ser mayor que 10"),  
    () -> assertTrue(resultado < 100, "Debería ser menor que 100")  
);
```

Cada comprobación es un `org.junit.jupiter.api.function.Executable` — con lambdas (como arriba) es la forma más compacta; también admite clases internas anónimas o clases con nombre si el `Executable` se va a reutilizar en varios tests, aunque para la mayoría de casos la lambda basta y sobra.

4. Ciclo de vida de un test

4.1. Una instancia por test (por defecto)

JUnit crea **una instancia nueva de la clase de test por cada método `@Test`**. Es lo que permite declarar el objeto a probar como atributo de instancia sin que un test pueda "contaminar" a otro con su estado:

```
public class CalculadoraTest {  
  
    private Calculadora calculadora = new Calculadora(); // uno nuevo por cada test  
  
    @Test  
    void test1() { /* ... */ }  
  
    @Test  
    void test2() { /* ... */ }  
}
```

Se puede cambiar este comportamiento con `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` sobre la clase, para reutilizar una única instancia entre todos los tests (el comportamiento clásico de JUnit 4). **No es recomendable** salvo que se tenga la certeza absoluta de que ningún test queda acoplado a otro a través de ese estado compartido — es una fuente clásica de tests que solo fallan si se ejecutan en un orden concreto.

4.2. Métodos de ciclo de vida

Anotación	Cuándo se ejecuta
<code>@BeforeAll</code> (método <code>static</code>)	Una única vez, antes de todos los tests de la clase. Tiene que ser <code>static</code> porque se ejecuta antes de que exista ninguna instancia de la clase de test (salvo con <code>PER_CLASS</code> , donde no hace falta que sea estático).
<code>@AfterAll</code> (método <code>static</code>)	Una única vez, después de todos los tests de la clase.
<code>@BeforeEach</code>	Antes de cada test. Es el sitio recomendado para inicializar el objeto a probar, en vez de compartirlo como atributo estático.

Anotación	Cuándo se ejecuta
@AfterEach	Después de cada test — típicamente para limpieza (cerrar recursos, borrar ficheros temporales...).

Puede haber más de un método marcado con la misma anotación (varios `@BeforeEach`, por ejemplo); JUnit los ejecuta todos, aunque no garantiza un orden concreto entre ellos si están en la misma clase.

Orden real de ejecución para una clase con dos tests:

```
beforeAll()
  → new Test() → beforeEach() → test1() → afterEach()
  → new Test() → beforeEach() → test2() → afterEach()
afterAll()
```

5. Probar que algo lanza una excepción

La forma recomendada es `Assertions.assertThrows`, con una lambda que envuelve la llamada que debería fallar:

```
CalculadoraException ex = Assertions.assertThrows(
    CalculadoraException.class,
    () -> calculadora.dividir(dividendo, divisor)
);
assertEquals("División por cero", ex.getMessage());
```

JAVA

`assertThrows` falla el test si el código **no** lanza ninguna excepción, o si lanza una de un tipo distinto al indicado. Si lanza la excepción esperada, la devuelve — así se puede seguir comprobando su mensaje, su causa o cualquier dato que lleve, como en la segunda línea del ejemplo.



Antes de que existiera `assertThrows` (y sigue siendo tentador reinventarlo), la forma manual era un `try/catch` con `Assertions.fail(...)` en la rama que no debería alcanzarse:

```
try {
    calculadora.dividir(dividendo, divisor);
    Assertions.fail("No ha lanzado la excepción esperada");
} catch (CalculadoraException e) {
    // esto es lo esperado
}
```

JAVA

Funciona, pero es más código, más fácil de escribir mal (por ejemplo, olvidando el `fail` en la rama feliz) y no da acceso directo a la excepción capturada. `assertThrows` hace lo mismo en una línea.

6. Comprobar que algo no tarda demasiado

```
Assertions.assertTimeoutPreemptively(
    Duration.ofMillis(2000),
    () -> servicio.operacionQueDeberiaSerRapida()
);
```

JAVA

Existen dos variantes:

Método	Comportamiento
<code>assertTimeout(duracion, ejecutable)</code>	Espera a que el código termine (aunque se pase del tiempo indicado) y entonces compara cuánto ha tardado. No interrumpe nada.
<code>assertTimeoutPreemptively(duracion, ejecutable)</code>	Ejecuta el código en otro hilo y lo interrumpe en cuanto se cumple el plazo, sin esperar a que termine por sí solo.

6.1. La alternativa declarativa: `@Timeout`

En vez de envolver el código a probar en una lambda dentro del cuerpo del test, `@Timeout` se coloca directamente sobre el método (o sobre la clase, para aplicarlo a todos sus tests):

```
@Test
@Timeout(value = 2, unit = TimeUnit.SECONDS)
void operacionQueDeberiaSerRapida() {
    servicio.operacionQueDeberiaSerRapida();
}
```

JAVA

Por defecto, `@Timeout` se comporta como `assertTimeout`, **no** como `assertTimeoutPreemptively`: espera a que el método termine (aunque se pase del tiempo) y entonces compara cuánto ha tardado — no interrumpe nada por sí solo. Para que sí interrumpa la ejecución en cuanto se cumple el plazo, hay que indicarlo explícitamente con `threadMode`:



```
@Test
@Timeout(value = 2, unit = TimeUnit.SECONDS, threadMode = Timeout.ThreadMode.SEPARATE_THREAD)
void operacionQueDeberiaSerRapida() {
    servicio.operacionQueDeberiaSerRapida();
}
```

JAVA

`@Timeout` también se puede fijar de golpe para todos los tests del proyecto, sin anotar cada método, con una propiedad en `junit-platform.properties` (ver Ejecución en paralelo para más propiedades de ese fichero):

```
junit.jupiter.execution.timeout.default = 2s
```

PROPERTIES



Un test que exige que algo se ejecute en un tiempo **concreto** es, en general, una mala idea fuera de un contexto muy controlado: obliga a que el test se ejecute siempre en un hardware al menos tan rápido como el usado para fijar ese límite. La misma máquina puede ir más lenta bajo carga, en una máquina virtual compartida, en un runner de CI saturado... así que este tipo de test es fácilmente inestable (*flaky*) aunque el código de producción no tenga ningún problema real de rendimiento.

Si el rendimiento importa de verdad, lo adecuado es medirlo con una herramienta de **benchmarking** dedicada (por ejemplo JMH), que está pensada para dar cifras fiables descontando el ruido del entorno — no con un test funcional que solo puede decir "sí" o "no" sobre un umbral arbitrario.

7. Tests parametrizados

`@ParameterizedTest` sustituye a `@Test` cuando el mismo cuerpo de test tiene que ejecutarse una vez por cada valor de una fuente de datos. Necesita el artefacto `junit-jupiter-params` (ver Dependencias Maven).

7.1. `@ValueSource` : una lista de literales

La fuente más simple: una lista de valores de un único tipo primitivo (o `String`).

```
@ParameterizedTest
@ValueSource(doubles = { 5, 25, 50, 75 })
void elCuadradoEsCorrecto(Double numero) {
    double resultado = calculadora.cuadrado(numero);
    assertEquals(numero * numero, resultado);
}
```

JAVA

Este test se ejecuta cuatro veces, una por cada valor de la lista, cada una contando como un test independiente en el informe.

7.2. `@MethodSource` : un método que genera los datos

Cuando los datos no son literales simples, o hacen falta varios parámetros a la vez, la fuente puede ser un método `static` de la propia clase que devuelva un `Stream<Arguments>` :

```
@ParameterizedTest(name = "El impuesto de {0} debe ser {1}")
@MethodSource("datosParaProbarImpuestos")
void elImpuestoEsCorrecto(Double ingreso, Double impuestoEsperado) throws InvalidIngresoException {
    double resultado = calculadoraImpuestos.calcularImpuestosSobreIngreso(ingreso);
    assertEquals(impuestoEsperado, resultado);
}

static Stream<Arguments> datosParaProbarImpuestos() {
    return Stream.of(
        Arguments.arguments(5000d, 0d),
        Arguments.arguments(10000d, 800d),
        Arguments.arguments(17000d, 1700d)
    );
}
```

JAVA



El método anotado con `@MethodSource` tiene que ser `static` (salvo que la clase use `@TestInstance(PER_CLASS)`) — si no, JUnit no puede llamarlo antes de crear una instancia de la clase de test.

El nombre pasado a `@MethodSource` es un `String` con el nombre del método — no hay comprobación en tiempo de compilación de que exista o de que la firma encaje, así que un error tipográfico ahí solo se detecta al ejecutar el test.

Los argumentos de `@MethodSource` no están limitados a tipos primitivos: pueden ser objetos completos del dominio (por ejemplo, instancias de una entidad), lo que permite parametrizar tests que necesitan un objeto de entrada complejo en vez de un puñado de valores sueltos.

7.3. Otras fuentes disponibles

`@ValueSource` y `@MethodSource` cubren la mayoría de casos, pero Jupiter trae más fuentes para situaciones concretas — vale la pena conocer que existen aunque no se usen todos los días:

Fuente	Para qué
<code>@CsvSource</code>	Datos tabulados directamente en la anotación, como líneas CSV ("5000, 0", "10000, 800").
<code>@EnumSource</code>	Recorre automáticamente los valores de un enum.
<code>@NullSource</code> / <code>@EmptySource</code> / <code>@NullAndEmptySource</code>	Añaden un caso con null, con una colección/cadena vacía, o ambos, sin tener que escribirlo a mano en la lista de valores.

8. Desactivar un test: `@Disabled`

`@Disabled` marca un test (o una clase entera, para desactivar todos sus tests de golpe) para que no se ejecute. A diferencia de un test que falla, un test desactivado aparece en el informe como **omitido**, no como fallo — el build no se rompe por su causa, pero tampoco desaparece sin dejar rastro.

```
@Test
@Disabled("Pendiente de arreglar: ver INCIDENCIA-123")
void calculoQueAunNoFunciona() {
    ...
}
```

JAVA

```
@Disabled("El endpoint que prueba esta clase todavía no existe")
class IntegracionConElNuevoServicio {
    // ningún test de esta clase se ejecuta
}
```

JAVA

El mensaje es opcional, pero es buena práctica ponerlo siempre: explica **por qué** está desactivado (referencia a una incidencia, motivo del bloqueo...) tanto en el informe de Maven como en el propio IDE, en vez de dejar que quien lo lea tenga que adivinarlo o rastrear el historial de Git.



`@Disabled` desactiva el test siempre, sin condiciones. Si lo que hace falta es desactivarlo solo en ciertas circunstancias (un sistema operativo, una variable de entorno, una versión de Java...), JUnit trae anotaciones declarativas para eso (`@EnabledOnOs`, `@DisabledOnOs`, `@EnabledIfSystemProperty` ...) que funcionan como una alternativa más legible a comprobar la condición a mano con `Assumptions` dentro del cuerpo del test (ver más abajo).



`@Disabled` no es lo mismo que quitar el `@Test` o comentar el método (`//@Test`), aunque el efecto práctico — que no se ejecute — sea el mismo. Un método sin `@Test` desaparece por completo para JUnit: no sale en ningún informe, y nada indica que ese test existió alguna vez ni por qué se dejó de ejecutar. `@Disabled` sí deja constancia explícita, con motivo incluido, de que ese test existe y está desactivado a propósito.

9. Precondiciones (Assumptions)

`Assumptions` expresa una **precondición** para que un test tenga sentido. Si no se cumple, el test no se da por fallido: se marca como **omitido** (`skipped`) en el informe.

```
@Test
void soloTieneSentidoEnLinux() {
    Assumptions.assumeTrue(
        System.getProperty("os.name").contains("Linux"),
        "Se omite: este test depende de una ruta específica de Linux"
    );

    // a partir de aquí, solo se ejecuta si la asunción se cumplió
    ...
}
```

JAVA

`Assumptions.assumingThat(condicion, ejecutable)` es la variante para condicionar solo **una parte** del cuerpo del test, en vez de omitirlo entero: el resto del método se sigue ejecutando normalmente.

Es el mecanismo adecuado para "este test solo aplica si..." (una variable de entorno, un sistema operativo, una característica opcional activada); no debe usarse como sustituto de `@Disabled` para silenciar un test que simplemente está roto.

10. Ejecución en paralelo

Por defecto, JUnit ejecuta todos los tests de forma secuencial, en un único hilo. Activar la ejecución en paralelo requiere dos cosas a la vez:

1. Habilitarla en `src/test/resources/junit-platform.properties`:

```
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = concurrent
junit.jupiter.execution.parallel.mode.classes.default = concurrent
```

PROPERTIES

2. Marcar qué debe ejecutarse en paralelo con `@Execution(ExecutionMode.CONCURRENT)`, sobre la clase o sobre un método suelto (o, como en el ejemplo anterior, dejar `concurrent` como modo por defecto para todo).



La ejecución en paralelo solo es segura si los tests **no comparten estado mutable** (un atributo `static`, un fichero, una fila de base de datos, una variable de entorno modificada por el propio test...). Si dos tests concurrentes tocan lo mismo, el resultado pasa a depender de qué hilo llegue antes — exactamente el tipo de test inestable que se quiere evitar. Antes de paralelizar una clase de test real hay que revisar primero que sus tests sean independientes entre sí; si no lo son, hay que marcarlos explícitamente con `@Execution(ExecutionMode.SAME_THREAD)`.

11. Extensiones (@ExtendWith)

Las anotaciones de ciclo de vida (@BeforeEach ...) cubren la mayoría de casos, pero cuando hace falta reutilizar un mismo comportamiento en muchas clases de test (o interceptar la ejecución de forma más fina), JUnit permite escribir **extensiones**: clases que implementan una o varias interfaces del paquete `org.junit.jupiter.api.extension`, cada una enganchada a un punto distinto del ciclo de vida.

Interfaz	Qué añade
BeforeAllCallback / AfterAllCallback	Equivalente programático a @BeforeAll / @AfterAll, pero reutilizable entre clases.
BeforeEachCallback / AfterEachCallback	Igual, para @BeforeEach / @AfterEach.
ExecutionCondition	Decide, en tiempo de ejecución, si un test debe ejecutarse o no (la base de mecanismos como @Disabled o @EnabledOnOs).
TestInstanceFactory / TestInstancePostProcessor	Controlan cómo se crea la instancia de la clase de test, o la modifican justo después de crearla (por ejemplo, para inyectarle dependencias).
InvocationInterceptor	El más potente: envuelve la llamada al propio método de test, pudiendo ejecutar código antes y después, o incluso decidir no llamar al test en absoluto.

Una extensión mínima que solo añade trazas alrededor de cada test:

```
public class TrazaExtension implements BeforeEachCallback, AfterEachCallback {  
  
    @Override  
    public void beforeEach(ExtensionContext context) {  
        System.out.println("Antes de: " + context.getTestMethod());  
    }  
  
    @Override  
    public void afterEach(ExtensionContext context) {  
        System.out.println("Después de: " + context.getTestMethod());  
    }  
}
```

JAVA

Y se activa sobre la clase de test con:

```
@ExtendWith(TrazaExtension.class)  
public class MiTest {  
    ...  
}
```

JAVA

Este mecanismo no es un detalle de bajo nivel poco usado: es la base de extensiones de terceros muy habituales en cualquier proyecto real, como `MockitoExtension` (para inyectar mocks con @Mock) o `SpringExtension` (para levantar un contexto de Spring en los tests de integración) — ninguna de las dos hace nada distinto de implementar estas mismas interfaces.

12. Buenas prácticas (y errores comunes)

- **Un test, un motivo para fallar.** Evitar que una misma clase de test mezcle pruebas de varias clases de negocio sin relación entre sí — dificulta saber, solo con el nombre de la clase que ha fallado, qué parte del sistema se ha roto.
- **Estructura Given/When/Then (Dado/Cuando/Entonces).** Preparar los datos, ejecutar la acción y comprobar el resultado como tres bloques separados y, si ayuda, comentados — hace el test legible incluso para alguien que no conoce el código de producción.
- **No compartir estado mutable entre tests.** Ni con atributos `static`, ni con `@TestInstance(PER_CLASS)` sin necesidad real: un test no debería poder influir en el resultado de otro.
- `equals()` **para valores**, `==` / `assertSame` **para identidad.** Especialmente con envoltorios (`Integer`, `Long` ...) y `double` / `float`.
- **Preferir `assertThrows` a un `try/catch` manual** para probar excepciones — es más corto y menos propenso a errores (como olvidar el `fail()` en la rama que no debería alcanzarse).
- **Cuidado con los tests dependientes del entorno de ejecución** (tiempo transcurrido, hora del sistema, orden de ficheros en un directorio, `Locale` por defecto...): son la causa más habitual de tests que fallan "sin motivo" en una máquina distinta a la del autor. `Assumptions` ayuda a omitir explícitamente lo que no aplica en el entorno actual, en vez de dejar que falle en rojo.
- **Los métodos `@MethodSource` deben ser `static`**, y su nombre se referencia como texto — un error tipográfico ahí no lo detecta el compilador, solo aparece al ejecutar el test.

13. Referencia rápida

Anotación	Uso
<code>@Test</code>	Marca un método como test.
<code>@DisplayName("...")</code>	Nombre legible para el informe.
<code>@BeforeAll</code> / <code>@AfterAll</code>	Una vez por clase (método <code>static</code> , salvo <code>@TestInstance(PER_CLASS)</code>).
<code>@BeforeEach</code> / <code>@AfterEach</code>	Antes/después de cada test.
<code>@TestInstance(Lifecycle.PER_CLASS)</code>	Reutiliza una única instancia de la clase de test entre todos sus tests.
<code>@Disabled("motivo")</code>	Desactiva un test (o una clase entera).
<code>@ParameterizedTest</code>	Ejecuta el mismo test una vez por cada valor de una fuente de datos.
<code>@ValueSource</code> / <code>@MethodSource</code> / <code>@CsvSource</code> / <code>@EnumSource</code>	Fuentes de datos para <code>@ParameterizedTest</code> .
<code>@Execution(ExecutionMode.CONCURRENT SAME_THREAD)</code>	Controla si una clase/método puede ejecutarse en paralelo.
<code>@Timeout(value = n, unit = ...)</code>	Alternativa declarativa a <code>assertTimeoutPreemptively</code> / <code>assertTimeout</code> .

Anotación	Uso
<code>@ExtendWith(MiExtension.class)</code>	Engancha una extensión a la clase de test.

Aserto / utilidad	Uso
<code>assertEquals</code> / <code>assertNotEquals</code>	Comparación por valor (<code>equals()</code>).
<code>assertTrue</code> / <code>assertFalse</code>	Cualquier expresión booleana.
<code>assertNull</code> / <code>assertNotNull</code>	Comprobación de <code>null</code> .
<code>assertSame</code> / <code>assertNotSame</code>	Comparación por identidad (<code>==</code>).
<code>assertArrayEquals</code> / <code>assertIterableEquals</code>	Comparación elemento a elemento.
<code>assertThrows(Tipo.class, () → ...)</code>	Comprueba que se lanza una excepción concreta.
<code>assertTimeoutPreemptively(duracion, () → ...)</code>	Comprueba (e interrumpe) que algo no tarda más de lo indicado.
<code>Assertions.fail("...")</code>	Falla el test explícitamente.
<code>Assertions.assertAll(() → ..., () → ...)</code>	Ejecuta varios asertos aunque alguno falle, informando de todos.
<code>Assumptions.assumeTrue(condicion, "...")</code>	Omite el test si no se cumple la condición.



Todos los conceptos de este manual se pueden practicar de forma guiada en la práctica `Ej01_JUnit6` del curso.